

CNCF Observability TAG Query Language

Standardization Working Group

DRAFT Semantic Specification

Status: **DRAFT**

Author(s): Chris Larsen

Last Updated: Apr 2, 2025

_ Goals	3
_ Data Models	4
_ Selection	33
_ Time Based Windowing	35
_ Aggregation	36
_ Joins	37
_ Query Limits	38
_ Cross Telemetry Retention Handling	38
_ Metric Rollup Handling	38
_ Functions	38

_ Goals

- Provide a set of standard telemetry models for querying observability data
- Provide a set of standard intermediate and output models for query results
- Define a set of semantic specifications for querying and analyzing observability telemetry
- Prioritize default behaviors that ease novice user understanding of the results
- Prioritize accuracy of results over precision. Avoid interpolation and extrapolation when possible.
- Interoperability with prevalent existing implementations
- Consistent behavior when querying data at rest or streaming events
- Metric coercion should behave the same regardless of source, i.e. metrics, events, logs, traces, profiles, etc.
- Reduce end user support burden on observability systems operators

_.1 Non Goals

- Define a syntax for the standard.
 - That will be for follow up work. Existing observability DSLs should be able to adopt the semantics for the most part.

- Define intermediate representations of query plans
 - This will also be follow up work and is happening in projects like Substrait.
- Support for fixed point numeric calculations, e.g. for financial analytics.
- Define a serialization output format.
- Define a physical layout of the data.

_.2 Context

For more context on the choices made in outlining this specification, see the

[CNCf DRAFT QLSWG Semantics Specification Context](#)

_.3 Working Group Recommendation

The recommendation of the CNCf working group is to follow the Relational Algebra model for observability as much as practically possible. Observability data is inherently relational and all of the popular implementations effectively deal with telemetry information as such.

The Structured Query Language is the most widely implemented, albeit imperfect or incorrect (at times), version of relational theory. Thus the recommendation laid out in this specification is:

- Adhere to SQL semantics as defined in ISO/IEC 9075 whenever possible.
- Adopt a subset of SQL types and operators pertaining to observability requirements.
- Specify table valued functions and common table expressions for interoperability.
- Features that cannot be handled via SQL semantics, define custom behaviors and lean on the wider SQL industry to support them.

This specification is NOT recommending the following:

- That every observability database add or implement a SQL engine. If an engine adheres to the *semantics* of this standard, then interoperability can be achieved by writing syntax translators and federated query engines that call into existing systems.
- That users adopt the SQL syntax or a dialect thereof. Semantic compatibility is the critical feature of this spec, not syntactic compatibility. There is still room for syntactic experimentation.

_ Data Models

SQL semantically operates on rows of records with various attributes. With decades of use, the standard has proven that it is capable of mapping to many different physical layouts of data. In the observability realm, metrics and logs have traditionally mapped cleanly to row based systems. Distributed traces and profiles are a bit more tricky.

Ordering of fields in models is undefined.

_.1 Data Types [[context](#)]

The following are data types supported by this specification.

_.1.1 Double [[context](#)]

Double precision floating point (IEEE 754-1985). Commonly used for metrics. Section 4.5 of ISO/IEC 9075-2:2023

_.1.2 Signed Integer [[context](#)]

Signed 64 bit integer. Used for metrics, sometimes for casting from strings to values. Section 4.5 of ISO/IEC 9075-2:2023

_.1.3 Unsigned Integer [[context](#)]

Unsigned 64 bit integer. Required for memory address referencing in profiles and stack traces. Section 4.5 of ISO/IEC 9075-2:2023

_.1.4 String [[context](#)]

Section 4.3 of ISO/IEC 9075-2:2023 with the following modifications:

- The only supported collation is UTF-8 ISO/IEC 10646.
- Implementations SHOULD prefer null values over empty strings.

_.1.5 Binary String [[context](#)]

A sequence of octets from section 4.4 of ISO/IEC 9075-2:2023

Note: We may leave out operators on binary strings initially, just support the type for serialization/deserialization purposes.

_.1.6 Boolean [[context](#)]

A value of either true or false. Section 4.6 of ISO/IEC 9075-2:2023

_.1.7 Timestamp [[context](#)]

Section 4.7 of ISO/IEC 9075-2:2023. A specific point in time.

- MUST support nanosecond resolution

_.1.8 Interval/Duration [[context](#)]

Section 4.7 of ISO/IEC 9075-2:2023. A range of time, i.e. a duration.

_.1.9 Map [[context](#)]

An unordered list of entries of key / value pairs.

__TODO__ Likely a UDT from section 4.9.3 of ISO/IEC 9075-2:2023? Probably don't want to mess with MULTISets.

- All keys MUST be of the same type
- All values MUST be of the same type
- NULL keys are allowed
- NULL values are allowed
- The ordering is non-deterministic **__TODO__** Should we order them?

_.1.10 Array [[context](#)]

A collection of data types with each element associated with exactly one ordinal position. Every entry in the array MUST be of the same type. From section 4.12.2 of ISO/IEC 9075-2:2023.

Supported types include:

- [String](#)
- [Double](#)
- [Signed Integer](#)
- [Unsigned Integer](#)
- [Binary String](#)
- [Boolean](#)

_.1.11 JSON Types [[context](#)]

From sections 4.8 and 4.48.3 of ISO/IEC 9075-2:2023. Used for logs, events, attribute values, etc. Coercion rules follow those of equivalent types above.

_.2 Attributes [[context](#)]

This standard uses the key/value convention with the name `attributes` following the [OpenTelemetry specification](#). It is also recommended, but not required, that users and systems follow the [OpenTelemetry Semantic Conventions](#) when naming attributes to enable cross telemetry correlation.

- Attributes for a record MUST NOT be NULL.
- Attributes for a record MAY be empty.

_.2.1 Keys

- A key MUST be a non-NULL and non-empty [String](#)
- Strings MUST contain ONLY printable characters
- Strings MUST NOT contain control characters (UTF8).
- Keys MUST be distinct
- Keys MUST be treated as case sensitive. Differing cases are treated as separate keys.

_.2.2 Values

- Values are typed and MUST be ONE of
 - [String](#)
 - [Double](#)
 - [Signed Integer](#)
 - [Unsigned Integer](#)
 - [Binary String](#)
 - [Boolean](#)
 - [JSON type](#) (differs from OTel spec)
 - NULL Required as some vendors allow for stand-alone strings instead of key/value pairs. (also allowed by W3C spec)
 - An [Array](#) of ONE of the following values
 - The array must be non-empty
 - Arrays MUST have one of the following types:
 - [String](#)
 - [Double](#)
 - [Signed Integer](#)
 - [Unsigned Integer](#)
 - [Binary String](#)
 - [Boolean](#)
 - [JSON type](#) (differs from OTel spec)

_.2.3 Coercion/Casting into Metrics

A common operation for telemetry is converting an attribute *value* to a metric, particularly for logs or spans. It usually happens with the values associated with a key, but some operations may coerce the count of keys, key substrings, or value types, etc.

- If a function extracts the value assigned to an attribute key:
 - The key string becomes the metric name unless the user provides an explicit override
 - The key and value is removed from the attribute list and remaining attributes are associated with the new metric

_.3.0 Metrics [[context](#)]

Metrics are numeric measurements captured as snapshots or summaries over a period of time. Early measurements were simply integer or floating point values but have now evolved into more complex types like min/max/sum/count summaries, histograms and digests.

The following is a summarization of the metric model. Each field links to a detailed description.

Field Name	Type	NULLable	Default
------------	------	----------	---------

start_timestamp	Timestamp	No	NA
end_timestamp	Timestamp	No	NA
attributes	Attributes	No	Empty
type	Enum: See entry	No	UNKNOWN
value	See entry	No	NA
step	Interval	Yes	NULL
temporal_aggregation	String	Yes	NULL
group_aggregation	String	Yes	NULL
temporal_boundaries	Array of timestamps	Yes	NULL
exemplars	Collection of Exemplars	No	Empty

- By default, metrics MUST be presented ORDERED by the “end_timestamp” ASCENDING. Ordering may be overridden.

Start Timestamp [\[context\]](#)

The exclusive start timestamp of the record, i.e. (start, end]

__TODO__ Still an open question if we want both start and end

Field Name	Type	Nullable	Default
start_timestamp	Timestamp	No	NA

- If no known start period exists, or the record is of a zero duration event (e.g. a metric gauge, an error message), the `start\_timestamp` MUST be the same as the `end\_timestamp`.

End Timestamp [\[context\]](#)

The inclusive end timestamp of the record. I.e. (start, end]

Field Name	Type	Nullable	Default
end_timestamp	Timestamp	No	NA

Attributes

The tags, labels, dimensions or keys/values associated with a record.

Field Name	Type	Nullable	Default
attributes	Attributes	No	Empty

Name

All metrics include a descriptive, typically human parseable, name of what is being measured. E.g. "system.cpu.busy.pct" or "ipc.server.call".

Field Name	Type	Nullable	Default
name	String	No	NA

- MUST be non-empty string
- MUST contain ONLY printable characters
- MUST NOT contain control characters.
- MUST be treated as case sensitive. Differing cases are treated as separate names.

Type [\[context\]](#)

The type field indicates the type of data stored to enable validation of functions.

Field Name	Type	Nullable	Default
type	Enum: See below	No	UNKNOWN

The type MUST be one of:

- **UNKNOWN** - If the type is unknown, this should be set. It is also the default (**__TODO__** Maybe UNSET to mimic other enums?)
- **CUMULATIVE_COUNTER** - A cumulative monotonic counter
- **DELTA_COUNTER** - a delta measuring the change between reporting/scraping intervals
- **GAUGE** - a state measurement that may fluctuate up or down in time
- **RATE** - a delta converted to a rate of increments per second
- **HISTOGRAM** - A bucketed histogram
- **APPROXIMATE_COUNT** - An approximate count structure
- **APPROXIMATE_DISTRIBUTION** - An approximate distribution structure
- **CUSTOM** - A non-standard data type for existing implementations that may not conform to the spec.

Value [\[context\]](#)

The value field represents the numeric measurement. It is commonly used implicitly in queries.

Field Name	Type	Nullable	Default
value	See rule below	No	NA

- The type MUST be one of
 - [\[Array of\] Double](#)
 - [\[Array of\] Signed Integer](#)
 - [\[Array of\] Binary String](#) (the type MUST be CUSTOM in this case)
 - [\[Array of\] Histogram](#)
 - [\[Array of\] Approximate Count](#)
 - [\[Array of\] Approximate Distribution](#)
- If the value is an array:
 - All non-NULL arrays in the record set SHOULD have the same number of entries. (**__TODO__** should queries mix different windowing intervals in the same result set?)
 - Values in the array MUST be ORDERED from the oldest to latest value.
 - NULLs should represent non-existent values within the arrays
 - The 'temporal_aggregation' field MUST NOT be NULL
 - The 'step' field MUST NOT be NULL

Step [\[context\]](#)

For data that has been temporarily aggregated, the step size, or duration, between values.

Field Name	Type	Nullable	Default
step	Interval	Yes	NULL

- MUST NOT be null if the type is an [Array](#) of values
 - IF the step is inconsistent across values in the array, e.g. calendaring issues mean the duration can be different across the values, then the step MUST be an interval of 0 AND the
 - ELSE the [Temporal Boundaries](#) field MUST be filled with timestamps.

Temporal Aggregation [\[context\]](#)

For data that has been temporally aggregated, the name of the aggregation used.

Field Name	Type	Nullable	Default
------------	------	----------	---------

temporal_aggregation	String	Yes	NULL
----------------------	------------------------	-----	------

__TODO__ Enum or string? We want some standards but there may be function names that are useful to emit.

Group Aggregation [\[context\]](#)

For data that has been grouped or spatially aggregated, the name of the aggregation used.

Field Name	Type	Nullable	Default
group_aggregation	String	Yes	NULL

__TODO__ Enum or string? We want some standards but there may be function names that are useful to emit.

Temporal Boundaries [\[context\]](#)

For data that has been temporally aggregated using a calendar or function that returns an inconsistently sized interval for each value, a collection of inclusive interval start timestamps, one for each entry in the value collection.

Field Name	Type	Nullable	Default
temporal_boundaries	Array of timestamps	Yes	NULL

- MUST be NULL if the the “values” is NOT an array
- MUST have the same number of entries as the “values” array if not Null

Exemplars

[Exemplars](#) are an optional collection of zero or more observations contributing to a metric. These are typically distributed trace spans associated with a metric interval for systems that support them.

Field Name	Type	Nullable	Default
exemplars	Collection of Exemplars	No	Empty

- Ordering is undetermined

The field name is `exemplars`, all lower case.

_.3.1 Exemplar [[context](#)]

Timestamp

The timestamp of the observation, e.g. the end of a span measurement or the time when a latency was added to the metric.

Field Name	Type	Nullable	Default
timestamp	timestamp	No	NA

- MUST fall within the (start_timestamp, end_timestamp] of the associated metric record
 - **__TODO__** If start_timestamp and end_timestamp for the parent row are the same, the exemplar timestamp MUST be the same as the parent end_timestamp

Value

The value of the measurement.

Field Name	Type	Nullable	Default
value	double	No	NA

Filtered Attributes

A set of attributes that were filtered out when aggregating into the parent metric.

Field Name	Type	Nullable	Default
filtered_attributes	See Attributes	No	Empty

- The filtered attributes MUST NOT contain any keys in the parent attributes map.

Trace ID

For logs that are reported within the context of a distributed trace, the trace ID should be set in this field.

Field Name	Type	Nullable	Default
trace_id	See Spans Trace ID	Yes	NULL

Span ID

For logs that are reported within the context of a distributed trace, the span ID should be set in this field.

Field Name	Type	Nullable	Default
span_id	See Spans Span ID	Yes	NULL

_.3.2 Histogram [[context](#)]

A bucketed histogram entry with descriptions and a list of values.

Buckets

The list of buckets.

Field Name	Type	Nullable	Default
buckets	Collection of Histogram Data Points	No	NA

- The buckets MUST be ordered from lowest lower boundary value to the highest upper boundary value.

Aggregation Temporality

The type of aggregation applied to the count of each histogram data point.

Field Name	Type	Nullable	Default
aggregation	Enum	No	UNSPECIFIED

- MUST be one of:
 - UNSPECIFIED - unknown or not set by the telemetry
 - DELTA - the count is reset between measurements
 - CUMULATIVE - the count is NOT reset between measurements

Description

An optional description of the type of histogram data underlying the model.

Field Name	Type	Nullable	Default
description	String	Yes	NULL

_.3.3 Histogram Data Point

Represents a set of data for a particular bucket in a histogram.

Count

The count of measurements falling within the bucket.

Field Name	Type	Nullable	Default
count	Signed Integer	No	NA

Lower Bounds

The inclusive lower boundary of the bucket, e.g. [lower, upper)

Field Name	Type	Nullable	Default
lower_bounds	Signed Integer or Double	No	NA

Upper Bounds

The exclusive upper boundary of the bucket, e.g. [lower, upper)

Field Name	Type	Nullable	Default
upper_bounds	Signed Integer or Double	No	NA

Sum

An optional sum of measurements in the bucket

Field Name	Type	Nullable	Default
sum	Signed Integer or Double	Yes	NULL

Min

An optional minimum measurement in the bucket.

Field Name	Type	Nullable	Default
min	Signed Integer or Double	Yes	NULL

Max

An optional maximum measurement in the bucket.

Field Name	Type	Nullable	Default
max	Signed Integer or Double	Yes	NULL

_.3.4 Approximate Count [[context](#)]

A type containing a probabilistic or approximate count of distinct objects.

Type

The type of structure.

Field Name	Type	Nullable	Default
type	String	No	NA

__TODO__ Should this be an enum? E.g. hll, datasketch, etc? Need standard names.

Data

The serialized payload.

Field Name	Type	Nullable	Default
data	Binary String	No	NA

_.3.5 Approximate Distribution [[context](#)]

A type containing a probabilistic or approximate distribution of measurements. Typically used for extracting quantiles.

Type

The type of structure.

Field Name	Type	Nullable	Default
type	String	No	NA

__TODO__ Should this be an enum? E.g. t-digest, data sketch, etc? Need Standard names.

Data

The serialized payload.

Field Name	Type	Nullable	Default
data	Binary String	No	NA

_.4 Logs/Events [[context](#)]

Logs map the cleanest to row semantics as each line of a log file corresponds to a row. The following is a summarization of the log model. Each field links to a detailed description.

Field Name	Type	Nullable	Default
start_timestamp	Timestamp	No	NA
end_timestamp	Timestamp	No	NA
attributes	Attributes	No	Empty
severity	Enum: See below	No	INFO
body	One of: • String • Binary String • JSON	No	NA
type	Enum: See below	No	String
trace_id	See Spans Trace ID	Yes	NULL
span_id	See Spans Span ID	Yes	NULL
fingerprint	String	Yes	NULL
sampling	Double	Yes	NULL

Start Timestamp

The exclusive start timestamp of the record, i.e. (start, end]

Field Name	Type	Nullable	Default
start_timestamp	Timestamp	No	NA

- If no known start period exists, or the record is of a zero duration event (e.g. a metric gauge, an error message), the `start\_timestamp` MUST be the same as the `end\_timestamp`.

End Timestamp

The inclusive end timestamp of the record. I.e. (start, end]

Field Name	Type	Nullable	Default
end_timestamp	Timestamp	No	NA

Attributes

The tags, labels, dimensions or keys/values associated with a record.

Field Name	Type	Nullable	Default
attributes	Attributes	No	Empty

Severity [\[context\]](#)

The severity or log level of the record. This typically reflects how impactful or urgent the message is. This spec will follow the [OpenTelemetry definitions of severity](#). Using an enum allows for common predicate comparisons like “severity > INFO”

Field Name	Type	Nullable	Default
severity	Enum: See below	No	INFO

MUST be one of

- TRACE - A fine grained event
- DEBUG - A debugging event
- INFO - Information that an event occurred
- WARN - An event that may require remediation and allows operations to continue
- ERROR - An event that should not have occurred and requires remediation but allows operations to continue.
- FATAL - An event that should not have occurred, requires remediation and prevented operations from continuing

Body [\[context\]](#)

The primary contents of the log entry. It can contain an unstructured string or a structured wide event.

Field Name	Type	Nullable	Default
body	One of:	No	NA

	• String • Binary String • JSON		
--	---	--	--

TODO - with OTel schemas, the attributes are the “wide events” essentially. BUT that precludes things like maps or complex data structures. So maybe schemas apply to the body too?

Wide events **MUST** be treated as JSON fields in the body, they **MUST** not be combined in attributes.

Type

The type field indicates the type of data stored to enable cleaner error handling.

Field Name	Type	Nullable	Default
type	Enum: See below	No	String

MUST be one of:

- STRING - A non-structured string
- BINARY STRING - A non-structured binary string
- JSON - A SQL JSON structure
- CUSTOM - A non-standard data type for existing implementations that may not conform to the spec.

Trace ID

For logs that are reported within the context of a distributed trace, the trace ID should be set in this field.

Field Name	Type	Nullable	Default
trace_id	See Spans Trace ID	Yes	NULL

Span ID

For logs that are reported within the context of a distributed trace, the span ID should be set in this field.

Field Name	Type	Nullable	Default
span_id	See Spans Span ID	Yes	NULL

Fingerprint [\[context\]](#)

Used to group a message based on commonalities. Generally used on unstructured log entries or exceptions.

Field Name	Type	Nullable	Default
fingerprint	String	Yes	NULL

Sampling

A probability score useful in estimating the frequency of sampled log events.

Field Name	Type	Nullable	Default
sampling	Double	Yes	NULL

- Value MUST be in the range [0.0 and 1]

_.5 Spans [\[context\]](#)

Traces are composed of spans, thus the span is the underlying model for this specification. Spans are a superset of log records with the addition of a parent/child relationship.

The following is a summarization of the span model. Each field links to a detailed description.

Field Name	Type	Nullable	Default
start_timestamp	Timestamp	No	NA
end_timestamp	Timestamp	No	NA
duration	duration	No	NA
attributes	Attributes	No	Empty
trace_id	String or Binary String	No	NA
span_id	String or Binary String	No	NA
parent_id	String or Binary String	Yes	NULL
kind	Enum: See below	No	INTERNAL

name	String	No	NA
state	Map	No	Empty
status_code	Enum: See below	No	UNSET
status_message	String	Yes	NULL
events	Collection of Span Events	No	Empty
links	Collection of Span Links	No	Empty
sampling	Double	Yes	NULL

Start Timestamp

The exclusive start timestamp of the record, i.e. (start, end]

Field Name	Type	Nullable	Default
start_timestamp	Timestamp	No	NA

- Though zero duration spans SHOULD be avoided, if the duration *is* zero, the `start\_timestamp` MUST be the same as the `end\_timestamp`.

End Timestamp

The inclusive end timestamp of the record. I.e. (start, end]

Field Name	Type	Nullable	Default
end_timestamp	Timestamp	No	NA

Attributes

The tags, labels, dimensions or keys/values associated with a record.

Field Name	Type	Nullable	Default
attributes	Attributes	No	Empty

Trace ID

The ID of the trace the span is associated with.

Field Name	Type	Nullable	Default
trace_id	String or Binary String	No	NA

- MUST be one of
 - A hexadecimal character string in lower case with 32 characters
 - A binary character string with 16 characters
- Must support automatic casting of character and binary between the backend implementation and output of the proper format.

Span ID

The ID of the span.

Field Name	Type	Nullable	Default
span_id	String or Binary String	No	NA

- MUST be one of
 - A hexadecimal character string in lower case with 16 characters
 - A binary character string with 8 characters
- Must support automatic casting of character and binary between the backend implementation and output of the proper format.

Parent ID

The parent ID of the span, pointing to the parent that may or may not exist in the table.

Field Name	Type	Nullable	Default
parent_id	String or Binary String	Yes	NULL

- MUST be non-empty and non-NULL IF the span is not a root span.
- MAY be NULL if the span is a root span.
- MUST be one of
 - A hexadecimal character string in lower case with 16 characters
 - A binary character string with 8 characters

- Must support automatic casting of character and binary between the backend implementation and output of the proper format.

Kind

The kind of the span when specified. Must follow the [Open Telemetry model](#).

Field Name	Type	Nullable	Default
kind	Enum: See below	No	INTERNAL

- This is an enumeration of valid types:
 - CLIENT - Outgoing request
 - SERVER - Incoming request
 - PRODUCER - Outgoing event
 - CONSUMER - Incoming event
 - INTERNAL - The default value representing an internal operation or unknown.

Name

The name of the span.

Field Name	Type	Nullable	Default
name	String	No	NA

Trace State

Vendor specific trace information, mapping to the Tracestate Header of the [W3C Trace Context](#) recommendation. Headers must be parsed into a collection of key/value pairs.

Field Name	Type	Nullable	Default
state	Map	No	Empty

- If the original header is truncated and a key does not have a value, the key MUST be dropped from the state.

Status Code

An enum representing whether the event measured by a span was successful or encountered an error.

Field Name	Type	Nullable	Default
status_code	Enum: See below	No	UNSET

MUST be one of:

- OK - A successful outcome
- ERROR - An errant outcome
- UNSET - The default value meaning the instrumentation library did not set a value.

Status Message

A descriptive message regarding the final state of the event measured by the span.

Field Name	Type	Nullable	Default
status_message	String	Yes	NULL

Events

Optional collection of zero or more events recorded during the span time frame.

Field Name	Type	Nullable	Default
events	Collection of Span Events	No	Empty

- Ordering is undetermined

Links

Optional links to spans in another trace.

Field Name	Type	Nullable	Default
links	Collection of Span Links	No	Empty

- Ordering is not determined

Sampling

A probability score useful in estimating the frequency of sampled traces (or spans).

Field Name	Type	Nullable	Default
------------	------	----------	---------

sampling	Double	Yes	NULL
----------	------------------------	-----	------

- Value MUST be in the range [0.0 and 1]

_.5.1 Span Events

Represents an annotation or event that occurred during the sampling.

End Timestamp

The end timestamp of the event.

Field Name	Type	Nullable	Default
end_timestamp	timestamp	No	NA

- MUST fall within the (start_timestamp, end_timestamp] of the associated span record
 - If start_timestamp and end_timestamp for the parent row are the same, the exemplar timestamp MUST be the same as the parent end_timestamp

Name

A description of the event.

Field Name	Type	Nullable	Default
name	String	No	NA

Filtered Attributes

A set of attributes that were filtered out when aggregating into the parent span.

Field Name	Type	Nullable	Default
filtered_attributes	See Attributes	No	Empty

- MUST not contain keys from the parent row attributes record

_.5.2 Span Links

A pointer from one span to another span in a separate trace.

Trace ID

The ID of the trace the span is associated with.

Field Name	Type	Nullable	Default
trace_id	See Spans Trace ID	No	NA

Span ID

The ID of the span.

Field Name	Type	Nullable	Default
span_id	See Spans Span ID	No	NA

Attributes

A set of attributes from the remote span.

Field Name	Type	Nullable	Default
attributes	See Attributes	No	Empty

Trace State

Trace state for the remote span.

Field Name	Type	Nullable	Default
state	See Trace State	No	Empty

5.3 Property Graph Modeling

The SQL 2023 ISO/IEC standard Part 16 describes support for property graphs. Modeling to support graph queries is as follows:

Vertex Table

- Table name = "spans"
- Key => "span_id" field
- Label => "span"
- Properties => ALL COLUMNS

Edge Table

- Table name = “traces”
- Key => “trace_id” field
- Source_key => “parent_id” REFERENCES spans (“span_id”)
- Destination Key “span_id” REFERENCES spans (“parent_id”)
- Label => “called”
- Properties => ALL COLUMNS

_.6 Profiles

Continuous profiles are modeled as a single sample per row.

Start Timestamp

The exclusive start timestamp of the record, i.e. (start, end]

Field Name	Type	Nullable	Default
start_timestamp	Timestamp	No	NA

- If no known start period exists, or the record is of a zero duration event (e.g. a metric gauge, an error message), the `start\_timestamp` MUST be the same as the `end\_timestamp`.

End Timestamp

The inclusive end timestamp of the record. I.e. (start, end]

Field Name	Type	Nullable	Default
end_timestamp	Timestamp	No	NA

Attributes

The tags, labels, dimensions or keys/values associated with a record.

Field Name	Type	Nullable	Default
attributes	Attributes	No	Empty

ID

A unique identifier for the profile that the samples belong to.

Field Name	Type	Nullable	Default
id	String	No	NA

- If no ID is provided at ingestion time, use a UUID for the profile.

Sample Type

The type of measurement the sample reflects. E.g. “cpu” or “syscall”

__TODO__ Should this be an enum

Field Name	Type	Nullable	Default
sample_type	String	No	NA

Sample Units

The units of measurement the sample reflects. E.g. “nanoseconds” or “count”

__TODO__ Undecided if this should be an enum

Field Name	Type	Nullable	Default
sample_units	String	No	NA

Temporal Type

The type of measurement, whether it is a delta or a cumulation.

Aggregation Temporality

The type of aggregation applied to the count of each histogram data point.

Field Name	Type	Nullable	Default
aggregation	Enum	No	UNSPECIFIED

- MUST be one of:
 - UNSPECIFIED - unknown or not set by the telemetry
 - DELTA - the count is reset between measurements
 - CUMULATIVE - the count is NOT reset between measurements

Frames

A list of function calls, thread paths, objects, etc.

Field Name	Type	Nullable	Default
frames	Collection of Frames	No	empty

- MUST be an ordered collection of type Frame **__TODO__ Which order?**

Value

The measurement.

Field Name	Type	Nullable	Default
value	Signed Integer or Double	No	NA

Trace ID

For profiles that are able to be captured in the context of a distributed trace, the trace ID should be set in this field.

Field Name	Type	Nullable	Default
trace_id	See Spans Trace ID	Yes	NULL

Span ID

For profiles that are able to be captured in the context of a distributed trace, the span ID should be set in this field.

Field Name	Type	Nullable	Default
span_id	See Spans Span ID	Yes	NULL

_.6.1 Frame

A frame of profile information.

Name

The human descriptive name of the frame.

Field Name	Type	Nullable	Default
name	String	No	NA

System Name

The system provided name.

Field Name	Type	Nullable	Default
system_name	String	No	NA

Attributes

Details about the frame.

Field Name	Type	Nullable	Default
attributes	See Attributes	No	Empty

Source Line

A reference to the source line if known.

Field Name	Type	Nullable	Default
source_line	Signed Integer	Yes	NULL

Source Column

A reference to the source column in a line if known.

Field Name	Type	Nullable	Default
source_line	Signed Integer	Yes	NULL

File Name

A reference to a file name if known and applicable.

Field Name	Type	Nullable	Default
file_name	String	Yes	NULL

Memory Start

A reference to the address at which the binary was loaded in memory, if applicable.

Field Name	Type	Nullable	Default
------------	------	----------	---------

memory_start	Unsigned Integer	Yes	NULL
--------------	----------------------------------	-----	------

Memory Limit

The limit of the address range the file occupies.

Field Name	Type	Nullable	Default
memory_limit	Unsigned Integer	Yes	NULL

File Offset

Offset in the binary that corresponds to the memory start..

Field Name	Type	Nullable	Default
file_offset	Unsigned Integer	Yes	NULL

Inlined

Whether or not the function was inlined, when the frame reflects a function call.

Field Name	Type	Nullable	Default
inlined	Boolean	Yes	NULL

__TODO__ Any other flags, columns?

_.7 Meta Data Models

A number of requests for observability data have to do with exploring the information available. E.g. list distinct metric names for an application or display the distinct attribute keys available. These do not require telemetry models. However, for situations where counts are sought, it is good practice to convert to metric models as they can then be charted by consumers.

__TODO__

_ Selection

_.1 Data Source

This specification is designed to enable joining and querying of observability data across different types of telemetry. The different types have traditionally been located in different data stores with varying implementations for efficiency reasons. E.g. metrics are often stored in a

dedicated time series database with logs in a document or RDBMS and traces in graph stores. In order for this specification to interoperate with existing tools, it must be able to specify the location of data.

SQL has the `FROM` clause to determine a table to select data from. Similarly this spec can use the same semantics to choose the telemetry type and enable federation across tables and independent stores.

The data source may be omitted when querying a specific store. E.g. Prometheus only deals with metrics and all data is available in a single view. So when sending a query to a Prometheus instance, no data source is necessary as the only source of data is implicit.

- The data source type definition, whether it contains metrics, logs, traces, etc., is an implementation detail.
- Sources MAY be fully qualified for federation.
- A data source MUST contain ONLY one type of telemetry data, i.e. metrics OR logs OR traces.

_.2 Predicates

Used to select a subset of data during a query.

- Implementations MUST follow SQL standards per
 - Section 8 Predicates ISO/IEC 9075-2:2023
 - Section 8 Predicates ISO/IEC 9075-16:2023
- Only those predicates listed below are REQUIRED by implementations.
- Predicands MUST be case sensitive equivalent matches by default.

_.2.1 Comparison Predicate

From Subsection 8.2 <comparison predicate> ISO/IEC 9075-2:2023 with the following modifications:

- Ordinal field positions are not allowed.

Implementations MUST support

- Equals
- Not equals
- Less than
- Less than or equal to
- Greater than
- Greater than or equal to

_.2.2 In Predicate

From Subsection 8.4 <in predicate> ISO/IEC 9075-2:2023

_.2.3 NULL Predicate

From Subsection 8.8 <NULL predicate> ISO/IEC 9075-2:2023

_.2.4 Similar Predicate

From Subsection 8.6 <similar predicate> ISO/IEC 9075-2:2023

Sometimes called `regexp`, this is the simplest form of regular expression support that observability tools support. Implementations MUST provide a subset of the POSIX regular expression syntax.

__TODO__ bring in the simple examples from Spectator

_.2.5 Period Predicate

From subsection 8.20 <period predicate> ISO/IEC 9075-2:2023

Note that we may save this for later versions. Few of the popular TSDBs support period predicates.

_.2.6 Source/Destination Predicate

From Subsection 8.4 <source/destination predicate> ISO/IEC 9075-16:2023

This is used for distributed traces.

_.2.7 CIDR Predicate

__TODO__ A means of filtering IPs by block.

_.3.0 Query Time Range Selection

[See context.](#)

The query time range refers to the overall time range for which a query is fetching data. The time range selection only applies to at-rest queries. Streaming queries do not require a time range.

- Implementations MUST allow for out-of-band specification of a time range
- Out-of-band predicates apply to the entire query
- Only one out-of-band query range can be supplied

- IF a query contains explicit time ranges, they MUST take precedence over out-of-band specified predicates.
- Both period and conditional predicates MAY be used for the time range selection
- Only results within the inclusive bounds of the query time range MAY be present in the output.
 - See [Time Based Windowing](#) for modifications
- Time based windows and functions MUST be allowed to modify the time range of data fetched and processed.

Time Based Windowing

A major aspect of observability querying deals with slicing a query time range into windows or buckets with the aim of reducing multiple data points into a single value. See [Context: Windowing](#) for important context as there are many differing opinions on this topic.

- Implementations MUST support normalization for all time based windows by default based on UTC boundaries.
- Windows MUST be [start, end), inclusive of the start timestamp and exclusive of the end timestamp. (Somewhat controversial from an SRE/telemetry engineering perspective, but expected from a *user* perspective)
- Implementations MUST accept a query spanning time zone specifier.
- If the start timestamp of the first interval is less than the query time range start timestamp, the first interval MUST be excluded.
- For the final interval in a query time range:
 - If the end of the final window is greater than or equal to the wall clock timestamp, the full data set of that interval MUST be evaluated and returned in the result.
 - If the end of the final window is less than the wall clock timestamp, the partial results MUST be returned.
- If the windowing specification is greater than the query time range, the result MUST be an empty result set with a warning stating the issue. **__TODO__ feedback system.**
- For duration units of a minute or less, the modulo method of normalizing MUST be used.
- For hour durations without a timezone, the modulo method MAY be used
- For hour durations with a timezone and intervals greater than an hour, normalization must happen based on Gregorian calendar alignment (**__TODO__ - find spec**)
 - E.g. 1 day, align on the start of a day, e.g. 00:00:00
 - E.g. 1 week, align on the start of Sunday
- The first interval MUST be normalized against the query window *start* time. Subsequent intervals are then incremented by step
- **__TODO__** - leap seconds + years
- **__TODO__** - DST

Implementations MAY provide a means of querying data without time based normalization but that aspect will not be part of this specification.

_ Aggregation

_.1.0 Math

In an effort to supply useful results, observability math typically follows slightly different rules.

- Not a number - NaN - MUST NOT be used as a sentinel value indicating the absence of data. NaN should only be emitted for strict mathematical operations that result in an invalid response.
- NULL MUST be used as the sentinel value indicating the absence of data.
- For addition and subtraction operations, the default behavior MUST be to substitute 0 for NULL.
- For multiplication, the default behavior MUST be to substitute a 1 for NULL.
- For division, the default behavior MUST be to substitute a 0 for a NULL in the divisor.
- **__TODO__ doublecheck**
- For averaging, NULLs MUST not increment the sum or count by default.

Implementations MUST also have differentiated functions that operate in a “strict” mathematical mode

_.2.0 Aggregating Names and Attributes

See [context](#).

_.2.1 Metric Names

- Implementations MUST offer a means of renaming a metric name. E.g. the ‘AS’ operator from SQL. Renaming takes precedence over other options.
- If multiple distinct metric names exist in an aggregated set, and no renaming is given, the aggregation and predicate should be used for the name.
- If an expression joins two or more metrics, and no renaming is given, a string representation of the expression SHOULD be used for the name.

_.2.2 Aggregating Attributes

- Only attribute keys present in a group by or join operator must be carried into the result set with the appropriate values.

_ Joins

A critical aspect correlating telemetry types and computing expressions on metrics is the ability to join data using common relations. The most common join will be on [attributes](#).

Semantics must follow subsection 7.10 <joined table> ISO/IEC 9075-2:2023 with the following modifications:

_.1.0 Default Joins

- For set to scalar joins, the default MUST be a cross join where the scalar is applied to every row in the set.
- For set to set joins, the default MUST be an inner equi-join on the `attributes` column.

_.1.1 Explicit Joins

- The implementations MUST allow for explicit joins
- Allowed explicit joins include:
 - Inner equi-join
 - Left outer equi-joins
 - Right outer equi-joins
 - Full outer equi-joins (or union)
- All joins must be equivalent joins for version 1.
- Cross joins are not allowed except on scalar to set operations.
- Natural joins are not allowed.

_ Query Limits

Observability systems are often in the critical path for operations in that users are using them to diagnose outages or tools use the data for scaling and auto remediation. Thus a number of implementations have configurable limitations on queries in order to prevent failures for all users. This specification lists some standard limitations that users may expect to encounter. Implementations may omit or add additional limitations.

__TODO__

_ Cross Telemetry Retention Handling

__TODO__

__ Metric Rollup Handling

Observability data is typically treated as an operation cost in businesses rather than a revenue source. Thus engineers have been pressed to reduce spend on data as much as possible and various strategies have been used to balance query latency with costs.

Metric queries on observability data are generally expected to be extremely fast, sub-second, and complete, no paging or incremental loading. To satisfy users, metrics are often temporally aggregated to lower resolutions over various durations. An early example is RRDTool's [Round Robin Archives](#) where users could specify various resolutions of data with different consolidation functions (min, max, avg, e.g.) and each write would update all of the resolutions at once. The RRDTool engine would choose the appropriate archive based on the query time range.

__TODO__

__ Functions

__TODO__ **Let's get the basics first.**